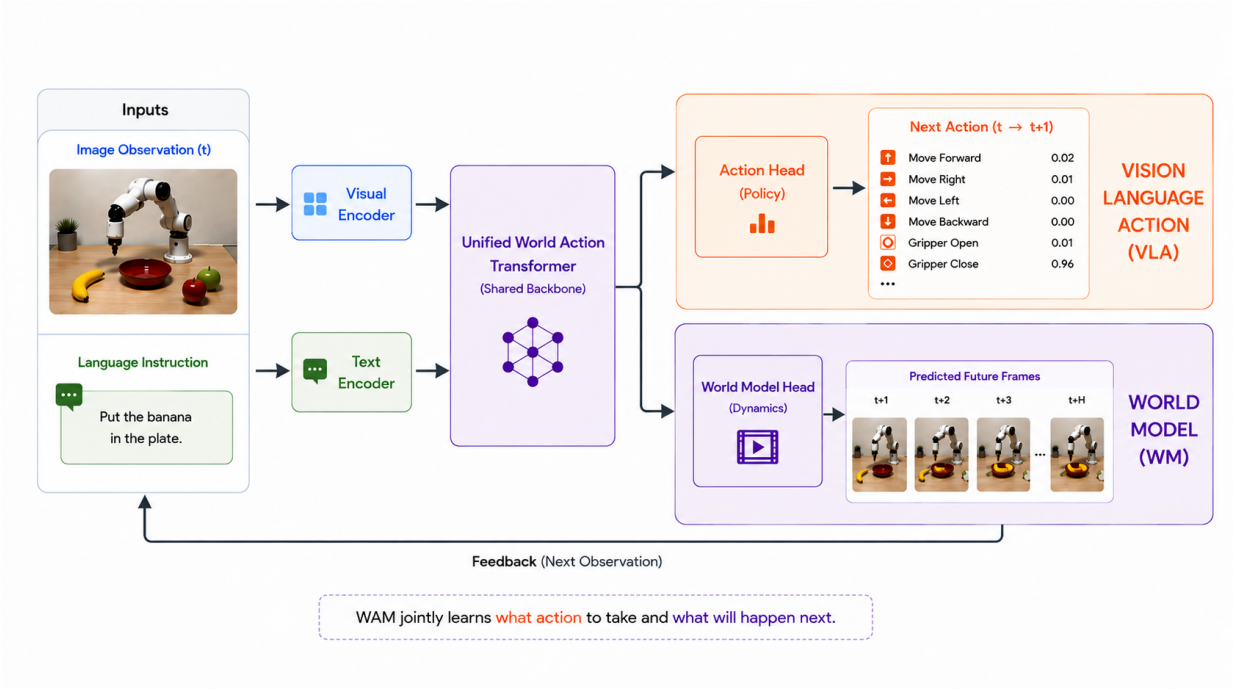
Video Action Models

Mikhail Toshchenko **Mikhail Orekhov** **Kirill Borodin**

Mentors: Nikita Kachaev · Artem Latyshev · Egor Cherepanov



Abstract

World Action Models (WAM) jointly predict future states of the environment and actions, and are expected to internalize the rules of the world through video pretraining. We test this assumption in a controlled setting: a compact NanoWM model is trained in environments built on XLand-MiniGrid and Baba-is-AI, where the rules of the dynamics are known by construction. The model does not bind rules to object appearance: on unseen color–shape combinations and on unseen rule compositions, the success rate is nearly identical to that on familiar tasks, and OOD performance grows monotonically with model size (0.81→0.92 from Small to Large); scaling the data shows a similar trend. The joint video+action objective gives no advantage when data is plentiful but shows a preliminary advantage when data is scarce; a moderate fraction of random trajectories in the training data does no harm; pretraining on related dynamics speeds up loss convergence without changing the final success rate. We also show that in environments with hidden rules, stochastic action decoding is critical: greedy decoding roughly halves the success rate even for a privileged expert.

1. Introduction

World Action Models (WAM) (Wang et al., 2026; Ye et al., 2026) are a rapidly growing family of models for agent control. They build on a pretrained video model and *jointly* predict future environment states and actions. As a result, already at the pretraining stage the model acquires a prior over world dynamics and the rules of interaction with objects, which allows it to generate more physically consistent actions and to generalize better than reactive VLA policies (Brohan et al., 2023; Kim et al., 2024; Physical Intelligence et al., 2025). However, what exactly such a model internalizes — the general *rules* that govern the world, or merely

the observed trajectories (Kang et al., 2024) — and what this depends on, remains an open question.

Testing such hypotheses on full-scale robotic WAMs is expensive: the models are heavy, training is slow, and the true laws of the real world are not available for direct comparison. Within the school, we therefore focused on a simplified setting, taking XLand-MiniGrid (Nikulin et al., 2024) as our testbed: it contains objects of different shapes and colors and explicit transformation rules (e.g., "yellow square + red circle → blue triangle"). In our ontology, the objects correspond to physical objects in the real world, and the transformation rules correspond to physical laws ("fire + paper → ash"). Since the rules are specified by construction, we can directly measure whether the world model has learned the rules and how it generalizes to novel combinations of them, while the GPU-accelerated environment and the compact NanoWM model (Huang et al., 2026) allow fast iteration over hypotheses.

In this setting we study five questions: **(Q1)** how the diversity of pretraining data affects quality and generalization; **(Q2)** whether failed and random trajectories are useful and how to use them properly; **(Q3)** how scaling laws behave for WAMs at small scale; **(Q4)** where the boundary lies between memorizing the dataset and learning the rules of the environment; **(Q5)** how the choice of training objective (joint video-and-action prediction versus actions only) affects the results.

Main findings. The model does learn rules rather than memorize trajectories: performance on unseen color–shape combinations and on unseen rule compositions matches that on familiar tasks within confidence intervals (Section 4.4), and OOD performance grows monotonically with model and data scale (Section 4.3). The joint video+action objective gives no advantage when data is plentiful; when data is scarce there is a trend in its favor that requires confirmation (Section 4.5). A moderate fraction of random trajectories (25%) does not hurt training; actions, however, should be learned from expert labels only (Section 4.2). Pretraining on related dynamics speeds up loss convergence but does not change the final success rate (Section 4.1). In addition, we show that in environments with hidden rules greedy action decoding systematically underestimates the model — this is a property of the environment, not of the model.

2. Related Work

Vision-Language-Action models. The dominant approach to generalist robot control is Vision-Language-Action (VLA) models such as RT-2 (Brohan et al., 2023), OpenVLA (Kim et al., 2024), and $\pi_{0.5}$ (Physical Intelligence et al., 2025): pretrained vision-language backbones are fine-tuned to map observations and language instructions directly to actions. VLAs inherit strong semantic generalization from web-scale pretraining, but they learn a *reactive* mapping $p(a \mid o, l)$ — they do not model how the world changes under the agent's actions. In practice this shows up as a need for many task-specific demonstrations and degraded performance out of distribution (OOD).

World Action Models. An emerging alternative integrates world models into action generation. Wang et al. (2026) formalize this paradigm as World Action Models (WAM): embodied foundation models that capture the *joint* distribution $p(o', a \mid o, l)$, coupling explicit prediction of future states with action generation. DreamZero (Ye et al., 2026) is a representative joint WAM: a video diffusion transformer that denoises future frames and actions together and acts as a *zero-shot policy*, roughly doubling zero-shot generalization to unseen tasks over comparable VLAs and transferring to new embodiments by fine-tuning on action-free video alone. Its results suggest that WAMs benefit from data *diversity* rather than the per-task repetition VLAs rely on.

Do predictive models learn the laws of the world? The WAM approach rests on a strong assumption: predicting the future forces the model to internalize the *rules* governing environment dynamics rather than memorize trajectories. Whether this actually happens is an open question. Kang et al. (2024) show that video models trained on simple physical scenes achieve near-perfect in-distribution prediction yet fail to recover the underlying physical laws: they exhibit *case-based* generalization, mimicking the closest training example instead of abstracting a general rule. The generalization ability of a WAM thus depends on whether it captures the laws that generate the observed dynamics rather than merely reproducing them — the question we study under controlled conditions.

Experimental substrate. In industry-scale WAMs, many design choices are entangled inside closed, heavyweight codebases, which makes controlled studies practically impossible. We therefore build on NanoWM (Huang et al., 2026) — a minimalist open implementation of action-conditioned video prediction based on diffusion forcing — and use XLand-MiniGrid (Nikulin et al., 2024) as our environment: a GPU-accelerated (JAX) gridworld with millions of procedurally generated tasks whose dynamics are governed by explicit compositional rules.

3. Problem Setting and Experimental Setup

3.1. Environment and Task

Our main environment is an XLand-MiniGrid configuration we call ShapeCraft-Easy-8×8 (Figure 1). The agent observes the full 8×8 board and chooses one of six actions

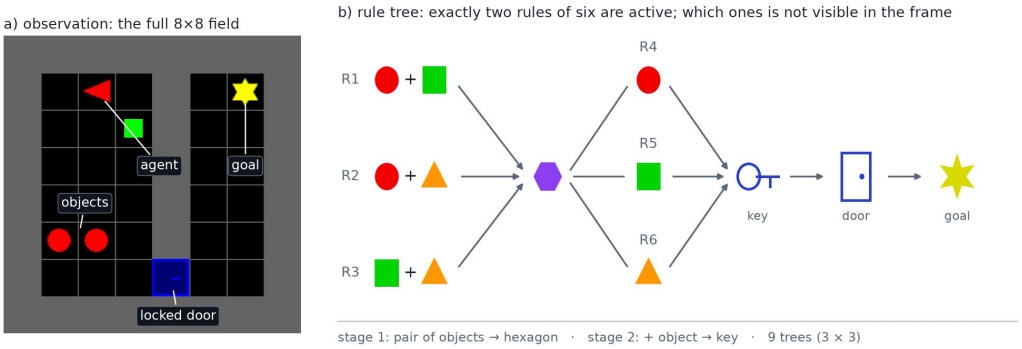


Figure 1. The ShapeCraft-Easy-8×8 environment: (a) observation — the full 8×8 board; (b) a rule tree of depth 2. Exactly two of the six rules are active in an episode; they are not visible in the frame and must be discovered by interaction.

(movement, turns, pick up / put down, open); transitions are deterministic, and the horizon is $H = 192$ steps. The reward is sparse and decays with time: $r_t = 1 - 0.9\,t/H$ if the goal is reached at step t, and 0 otherwise. The environment rule: two adjacent items are replaced by a third one; a rule tree of depth 2 leads to a key that opens a door hiding the goal. Two of the six rules are active and are not shown in the frame — the model must discover them by interacting.

For the random-trajectory experiment (Section 4.2) we additionally use the Baba-is-AI environment (puzzles with modifiable rules, 4 action classes); for the transfer experiment (Section 4.1) we use variants of XLand-MiniGrid dynamics: *exact* (standard movement), *stride2* (the agent moves two cells per step), and *omni8* (interaction along diagonals as well).

3.2. Data

Labels are provided by a privileged PPO policy: it has access to the active rules, the crafting stage, and the valid-action mask; the world model gets none of these. The policy solves 0.96 of training tasks and 0.86 of held-out tasks. The dataset consists of episodes of frames and actions with a window of 4 frames (3 transitions). Three sets with an equal per-task episode quota: training, a control set on the same tasks with new item layouts (val_id), and a held-out set (val_ood).

3.3. Model and Training

A frozen SD-VAE (fp32) encodes a 128×128 frame into a $4 \times 16 \times 16$ latent; with patch size 4 this yields 16 tokens per frame. The main model is NanoWM-S/4: 12 layers, width 384, 6 heads, 39.6M parameters; frames are predicted by diffusion (1000 steps, v-prediction, cosine noise schedule). The action head is a transformer: 3 transition queries, 2 layers, 4 attention heads, hidden size 256, dropout 0.1, 6 action classes; its loss weight is 1.0. Shared hyperparameters across all runs: batch size 8, 60,000 steps, lr 10^{-4}, weight decay 0.01, 500 warmup steps, gradient clipping at norm 1.0, fp16, seed 3407. The scaling experiments use three backbone configurations: Small (the main model above), Base (12 layers, width 768, 12 heads, 158M parameters), and Large (24 layers, width 1024, 16 heads, 557M parameters). In Baba-is-AI we use NanoWM-S/2 with an MLP action head (hidden size 256), 20,000 steps, and a 1-frame window.

3.4. Metric

Success rate (SR) in closed loop: the first of the three predicted actions is executed, then the model replans; $\mathrm{SR} = \frac{1}{N}\sum \mathbb{1}[\text{goal reached}]$, $N = 200$, on the training and held-out task banks; the PPO expert's SR serves as a ceiling. Actions are sampled with temperature T ($T = 0$ is argmax). Pixel metrics are not used: the agent occupies about 1.5 % of the frame, and reconstruction quality is only weakly related to task success.

4. Results

The results are grouped by the five questions from the introduction.

4.1. Q1: Diversity of Pretraining Data

Data volume. We trained the Base model on 1k, 3k, 10k, 30k, and 100k successful PPO trajectories (Figure 2). On OOD trees at $T = 1$, SR grows monotonically: from 0.80 (1k) to 0.97 (100k) for video+action, and from 0.67 to 0.90 for action-only. More diverse trajectories consistently improve OOD performance; no saturation is visible at the scales studied.

Pretraining on related dynamics. Pretraining for 30,000 steps on the stride2 and stride2+omni8 environments followed by fine-tuning on exact speeds up loss convergence (including the action loss), but does not change the final SR: the scratch and pretrained curves converge to the same level (Figure 3). At the scale studied, transfer from related

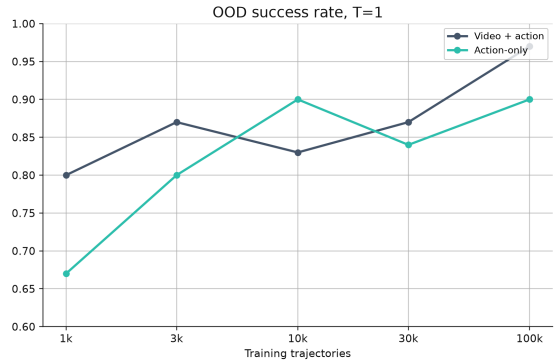


Figure 2. OOD success rate as a function of the number of training trajectories ($T = 1$). The 1k–10k and 100k points are evaluated on 30 episodes after 15,000 training steps; the 30k point comes from the main evaluation (200 episodes, 60,000 steps).

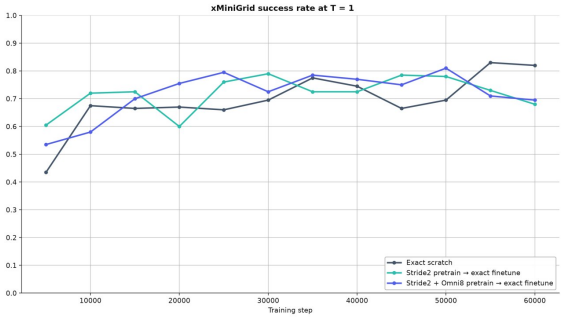


Figure 3. Success rate when fine-tuning on the exact task ($T = 1$): from scratch, after stride2 pretraining, and after stride2+omni8 pretraining. Pretraining speeds up loss convergence, but the final SR is comparable.

dynamics buys training speed, not final quality.

Data composition. The random-trajectory experiments (next section) show that state coverage can also be increased with non-expert data, but under a fixed training budget this is not free; the recipe that works is the two-phase scheme "world first, actions second."

4.2. Q2: Are Failed and Random Trajectories Useful?

Mixing in random trajectories (Baba-is-AI). Hypothesis: random trajectories cover more states and therefore improve the world model. Three runs with PPO/random trajectory ratios of 100/0, 75/25, and 25/75 (5000 episodes in total); on random trajectories the action loss is zeroed out so that the model does not imitate random actions. Result (Figure 4): the 75/25 mixture tracks the pure-expert curve (SR $\approx$ 0.6–0.8 by the end of training), while 25/75 consistently lags behind. Caveat: this is a single training seed and the curves are noisy (up to $\pm$0.15 between adjacent points), so the conclusion is conservative — a moderate fraction of random

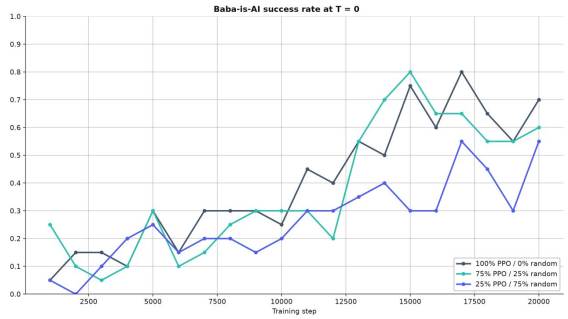


Figure 4. Baba-is-AI: success rate over training for three data compositions (PPO/random trajectory ratio).

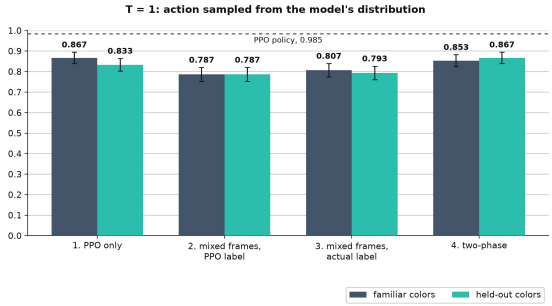


Figure 5. ShapeCraft: SR at $T = 1$ for four ways of collecting training data, on familiar (val_id) and held-out (val_ood) color–shape combinations. The dashed line is the PPO expert (ceiling).

data *does no harm*, but random trajectories do not replace expert ones.

Data composition for the world versus for actions (ShapeCraft). Four ways to collect training data, all else being equal (Figure 5): (1) expert states and actions only: 0.867/0.833 (val_id/val_ood); (2) 50/50 states from a random policy, actions labeled by the expert: 0.787/0.787; (3) the same frames with the actually executed actions: 0.807/0.793; (4) two-phase, 15,000 steps on random data, then a fresh action head trained for another 15,000 steps on PPO data: 0.853/0.867. Conclusion: mixing in random *states* under a fixed budget slightly degrades performance, but the two-phase "world first, actions second" scheme fully recovers it to the expert-data level. Failed trajectories are useful for training the world model, while actions are best learned from expert labels.

4.3. Q3: Scaling Laws at Small Scale

Three model sizes — Small, Base, Large — on fixed data (Figure 6). At $T = 1$, OOD SR grows monotonically with size: 0.81/0.84/0.92 for action-only, 0.76/0.86/0.89 for video+action. Together with the data-scaling results (Figure 2), this shows that at small scale a WAM benefits from both larger models and more data, and specifically

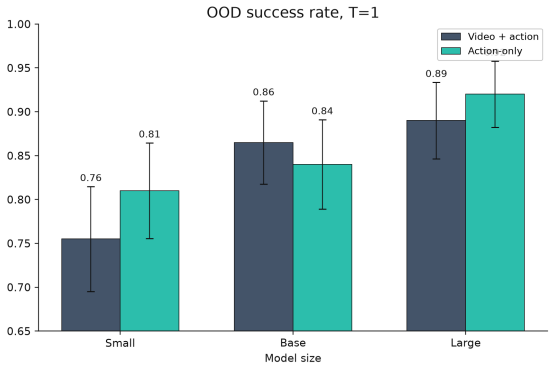


Figure 6. OOD success rate for Small, Base, and Large at $T = 1$. Whiskers are 95 % confidence intervals, $N = 200$.

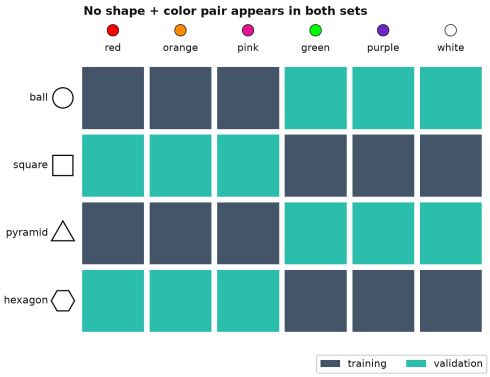


Figure 7. The color–shape split in ShapeCraft: training and validation use the same six colors and four shapes, but no color–shape pair appears in both sets.

on OOD tasks: scale converts into generalization, not just memorization. At $T = 0$ the trend persists, but the estimates are much noisier (0.42–0.55, Figure 9 in the appendix) and we do not recommend drawing conclusions from them (see Section 4.4).

4.4. Q4: Memorization or Rule Learning?

Rules are not bound to color. In ShapeCraft, transformations are determined by the *shapes* of items; color is pure decoration. Training and validation tasks are built from the same six colors and six rules, but the color–shape combinations do not overlap (Figure 7): if during training a ball is red, orange, or pink, then at validation it is green, purple, or white. Failure cannot be attributed to an unfamiliar color: the only unfamiliar thing is that this color appears on a ball. Result: there is no gap between familiar and held-out bindings in any measurement (Figure 5) — the model learned the rule as a property of shape, not of the appearance seen during training.

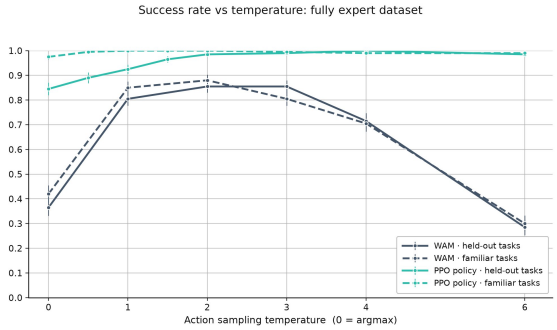


Figure 8. SR versus action-sampling temperature (model trained on expert data). $T = 0$ traps both the model and the PPO expert in loops: hidden rules require probing.

Familiar rules in novel compositions. The ID split contains new seeds of the six task trees present during demonstration collection; the OOD split contains three entirely held-out trees: the primitive rules are the same, but their order and the dependencies between subtasks change. At $T = 1$, over 200 closed-loop episodes, OOD is close to ID: 0.855/0.810 (Small), 0.860/0.840 (Base), 0.945/0.920 (Large) for action-only; 0.730/0.755, 0.875/0.865, 0.940/0.890 for video+action. There is no systematic collapse: differences of a few percentage points are comparable to the 95 % confidence intervals. The model applies familiar primitive rules in novel compositions rather than reproducing seen trajectories.

Action decoding matters more than data composition. Greedy action selection ($T = 0$) roughly halves the SR: for the model trained on expert data, 0.42 at $T = 0$ versus $\approx$0.88 at the optimal $T = 2$ (Figure 8); for the model trained on mixed data, 0.47 versus 0.905 (Figure 9 in the appendix). The same happens to the privileged expert (0.845 at $T = 0$ versus 1.000) — so this is a property of the environment, not of the model: the rules are hidden, they must be probed, and a deterministic policy gets stuck in loops. Under greedy rollout, all 664 out of 664 recorded failures hit the step limit. Practical takeaway: WAMs in environments with hidden rules must be evaluated with stochastic decoding (our main measurements use $T = 1$; SR peaks around $T \approx 2$), otherwise the evaluation underestimates the model.

4.5. Q5: Effect of the Training Objective

We compare two regimes: *video+action* (joint diffusion of future frames and actions — the WAM proper) and *action-only* (only the action head on top of the same backbone). With full data, joint prediction shows no advantage: action-only is comparable at Base and better at Large (Figure 6). With scarce data, however, the trend reverses: at 1k trajectories video+action reaches 0.80 versus 0.67 for action-only, and the gap narrows as the dataset grows (Figure 2). Caveat:

these points are evaluated on 30 episodes (95 % CI on the order of ± 0.15), so the difference is borderline significant and requires confirmation under the full protocol. If confirmed, the trend is consistent with the motivation behind WAMs: future-state prediction acts as an auxiliary signal that is most valuable when there are few actions to imitate. When expert data is plentiful, the extra video loss does not pay off at the scale studied.

5. Discussion

Taken together, the results support the core assumption of WAMs in our controlled setting: the model learns rules of the environment that are invariant to object appearance (Q4, colors) and transfer to novel compositions (Q4, trees), and rule learning improves with model and data scale (Q3, Q1). Two results refine this picture. First, the benefit of video prediction is not unconditional: it is an auxiliary signal that pays off when data is scarce (Q5). Second, WAM evaluation is sensitive to the protocol: greedy decoding in environments with hidden rules halves the measured quality of both the model and the expert (Q4) — when comparing models, this can completely mask real differences.

6. Limitations

(1) The data-volume dependence (Figure 2) is a preliminary trend, not a rigorous scaling law: the 1k–10k and 100k points are evaluated on 30 episodes with a smaller training budget than the 30k point; a rigorous conclusion requires equal compute, 200 episodes, and several seeds. (2) All findings are obtained in deterministic gridworlds with discrete actions; transfer to continuous robotic environments has not been tested. (3) The multi-embodiment experiments are not finished and are not included in this report. (4) A single training seed (3407) was used for the main runs, so small differences (e.g., ID versus OOD gaps of a few percentage points) should be interpreted with caution.

7. Conclusion

In a controlled setting built on XLand-MiniGrid and Baba-is-AI, we showed that a compact WAM learns the rules of the environment rather than memorizing trajectories: performance is preserved on unseen feature combinations and unseen rule compositions, and grows monotonically with scale. Practical recommendations: the world model can be trained on non-expert data as well (a moderate fraction of random trajectories is safe; the "world first, actions second" scheme works), while actions should be learned from expert labels; the video objective is worth enabling when data is scarce; evaluation should use stochastic decoding. Future work includes transferring these results to more complex robotic environments with pretraining on unlabeled human data, completing the multi-embodiment experiments, and running careful scaling studies with several seeds.

References

Brohan, A., Brown, N., Carbajal, J., et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

Huang, S., Kaushik, P., Chen, M., Pan, H., Geng, K., Chehab, O., Moreno-Pino, F., and Simchowitz, M. Nano world models: A minimalist implementation of future video prediction. *arXiv preprint arXiv:2605.23993*, 2026.

Kang, B., Yue, Y., Lu, R., et al. How far is video generation from world model: A physical law perspective. *arXiv preprint arXiv:2411.02385*, 2024.

Kim, M. J., Pertsch, K., Karamcheti, S., et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

Nikulin, A., Kurenkov, V., Zisman, I., Agarkov, A., Sinii, V., and Kolesnikov, S. XLand-MiniGrid: Scalable meta-reinforcement learning environments in JAX. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

Physical Intelligence, Black, K., Brown, N., et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

Wang, S., Shi, J., Fu, Z., et al. World action models: The next frontier in embodied AI. *arXiv preprint arXiv:2605.12090*, 2026.

Ye, S. et al. World action models are zero-shot policies. *arXiv preprint*, 2026.

A. Additional Plots

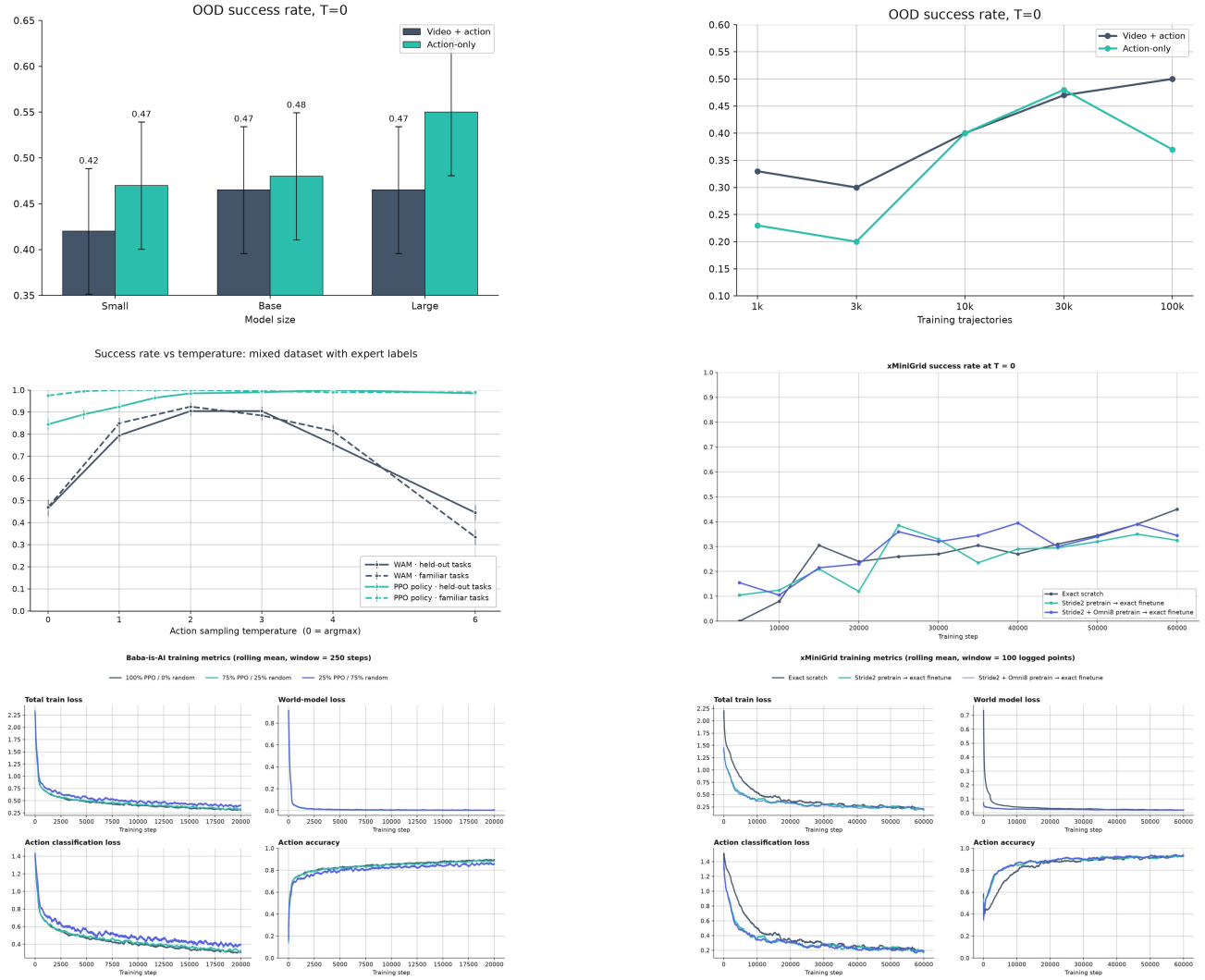


Figure 9. Additional plots. **Top:** evaluations under greedy decoding ($T = 0$) — OOD SR by model size (left) and by data volume (right); the estimates are substantially noisier and lower than at $T = 1$ (see Section 4.4). **Middle:** SR versus temperature for the model trained on mixed states with expert labels (left); fine-tuning on exact at $T = 0$ (right). **Bottom:** training curves — Baba-is-AI with random-trajectory mixing (left) and pretrained/scratch runs on xMiniGrid (right); pretraining speeds up the convergence of the action loss and action accuracy.